



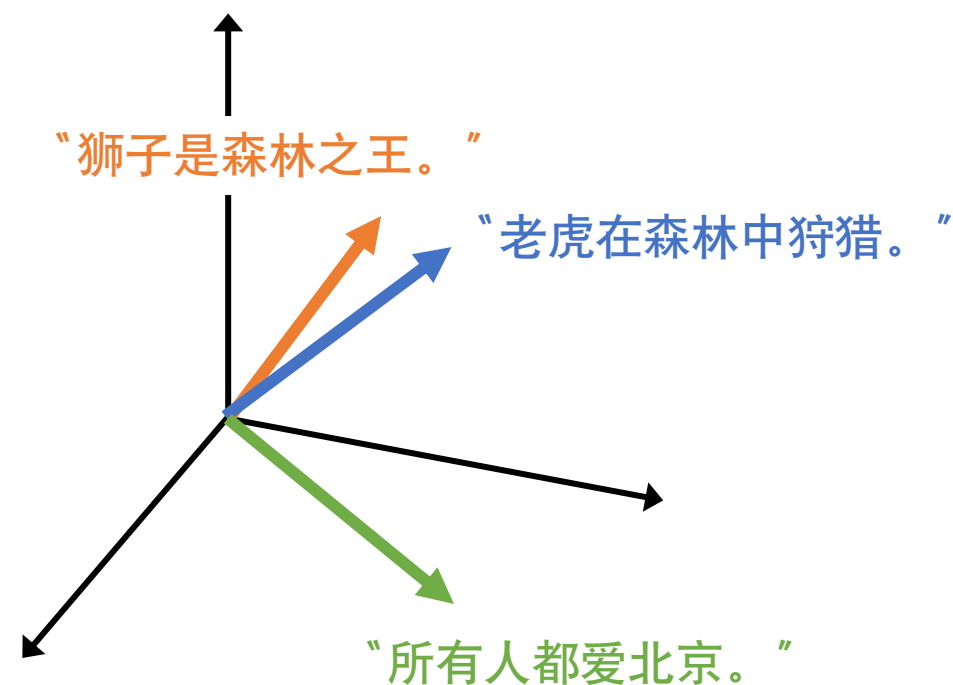
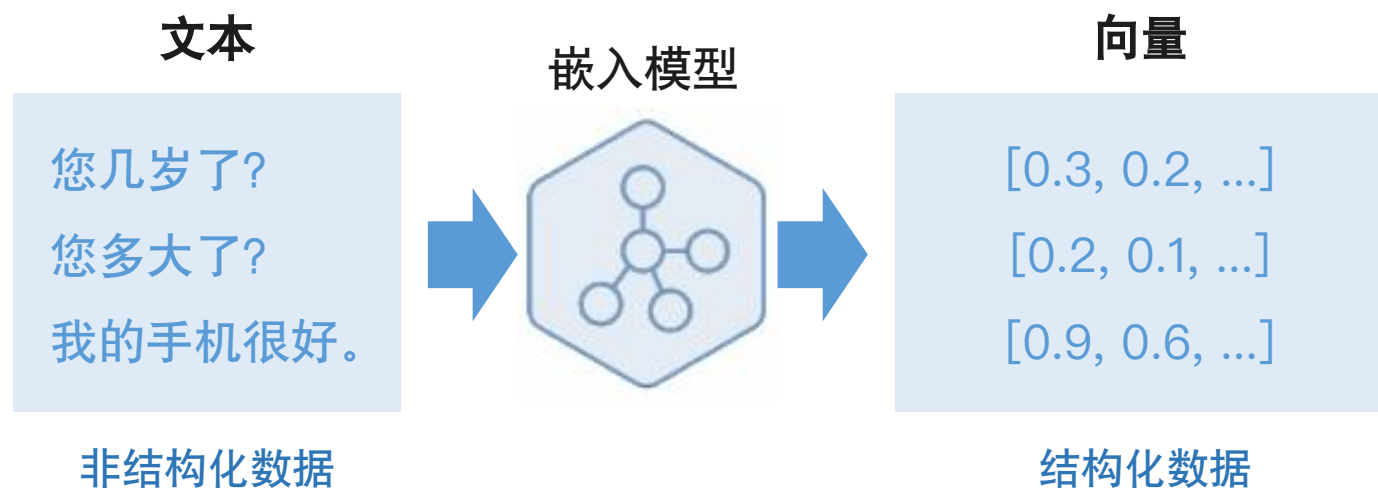
文本表示与理解

主讲人：苏勤亮



文本表示的目标

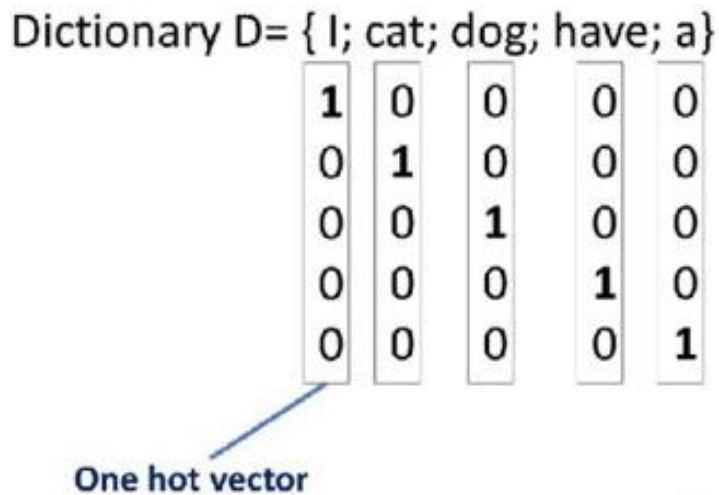
将文本映射为向量，向量间的某种距离能够反映语义相似性



大纲

- 基于词频的文本表示
- Word2Vec词嵌入
- 文本理解的预训练模型BERT
- 基于BERT 微调的文本理解

- 表示文本数据最简单的方式，是用一个 one-hot 向量来编码词汇表中的每个词，例如



- 之后，句子被表示为 one-hot 向量的连接

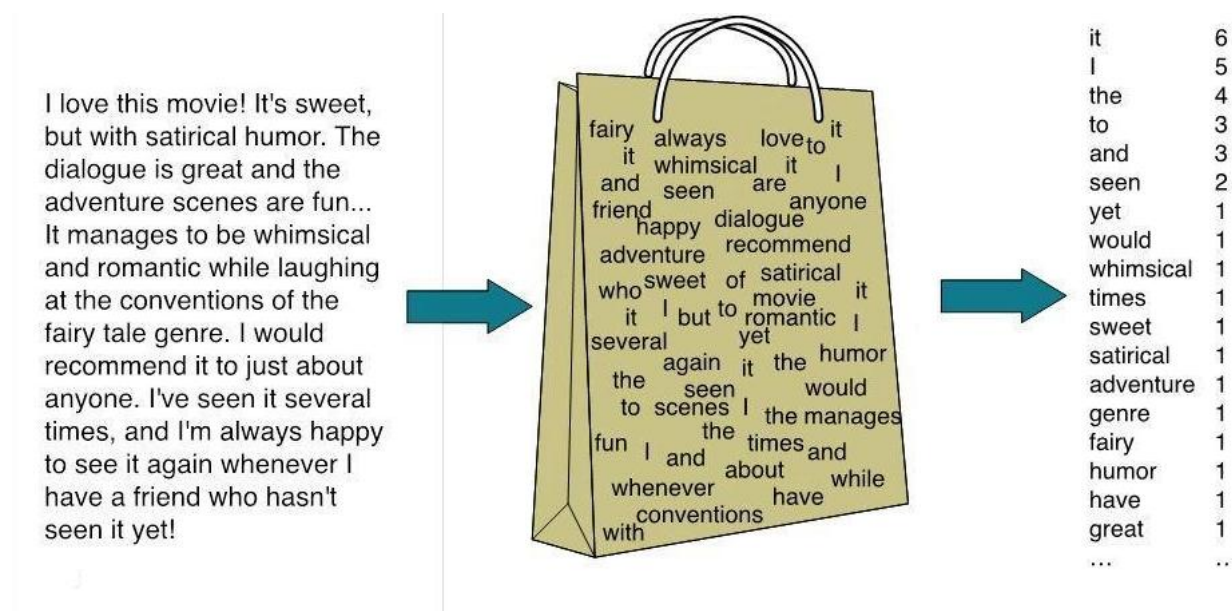
例如，“I have a dog” 可以表示为右边的矩阵

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

- 表示句子为一个矩阵是低效的，尤其是当句子很长的時候

词袋 (BOW)

词袋 (BOW) 丢弃了词语顺序的信息，只记录一个文档中每个词的计数



- **缺点：** 每个词都被认为具有同等的重要性。然而，不同词语所传达的信息量是不相等的

例如，像 'the', 'a' 这样的词语的信息量远不如像 'classroom', 'football' 等词语

- Term Frequency – Inverse Document Frequency (TFIDF) 试图反映不同词语的不等重要性
具体来说，它是通过两个统计数据的乘积计算得出的，即：

$$tfidf(w, d, \mathcal{D}) = tf(w, d) \times idf(w, \mathcal{D})$$

- w 表示词语； d 表示第 d 个文档； $\mathcal{D}$ 表示语料库（文档的集合）
- $tf(w, d)$ 表示词语 w 在文档 d 中的频率

$$tf(w, d) = \frac{\text{\# of word } w \text{ in } d}{\text{\# of all words in } d}$$

- $idf(w, \mathcal{D})$ 是语料库 $\mathcal{D}$ 中包含词语 w 的文档比例的对数

$$idf(w, \mathcal{D}) = \log \frac{\text{\# of documents in } \mathcal{D}}{\text{\# of documents containing word } w \text{ in } \mathcal{D}}$$

- $idf(w, \mathbb{D})$ 衡量了词语 w 包含多少信息。 $idf(w, \mathbb{D})$ 的值越大，词语的信息量就越大
- 因此，TFIDF 特征 $tfidf(w, d, \mathbb{D}) = tf(w, d) \times idf(w, \mathbb{D})$ 同时考虑了词语在文档中的频率和其在语料库中的信息量

$$tfidf(t, d, D) = tf(t, d) \cdot idf(t, D)$$

		blue	bright	can	see	shining	sky	sun	today
Document 1	1	0.301	0	0	0	0	0.151	0	0
Document 2	2	0	0.0417	0	0	0	0	0.0417	0.201
Document 3	3	0	0.0417	0	0	0	0.100	0.0417	0
Document 4	4	0	0.0209	0.100	0.100	0.100	0	0.0417	0

每一行代表一个文档的 TFIDF 特征

局限性

- One-hot 词语表示

- 无法反映词语的语义相似性，例如 'football' 和 'soccer'
- 当表示一个文档时，过于低效
- 维度非常高，高达 20000
- 非常稀疏

- BOW 和 TFIDF 文档表示

- 无法反映词语的语义相似性
- 不包含任何词语顺序信息
- 维度非常高
- 非常稀疏



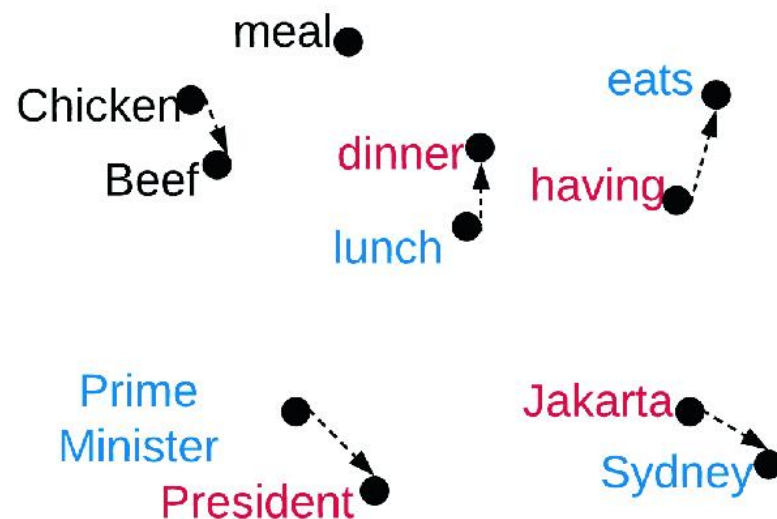
但比 one-hot 表示好得多

大纲

- 基于词频的文本表示
- Word2Vec词嵌入
- 文本理解的预训练模型BERT
- 基于BERT 微调的文本理解

Word2Vec 词嵌入表示

- 目标：用具有以下特征的实值向量表示单词
 - 1) 稠密 (低维度)
 - 2) 反映单词之间的语义相似性



- 句子嵌入可以随后从单词嵌入中构建

如何学习 Word2Vec 嵌入？

- **观察：** 单词的语义相似性隐含地反映在现有句子中

例如， 如果两个词经常同时出现，它们可能会保留相似的含义

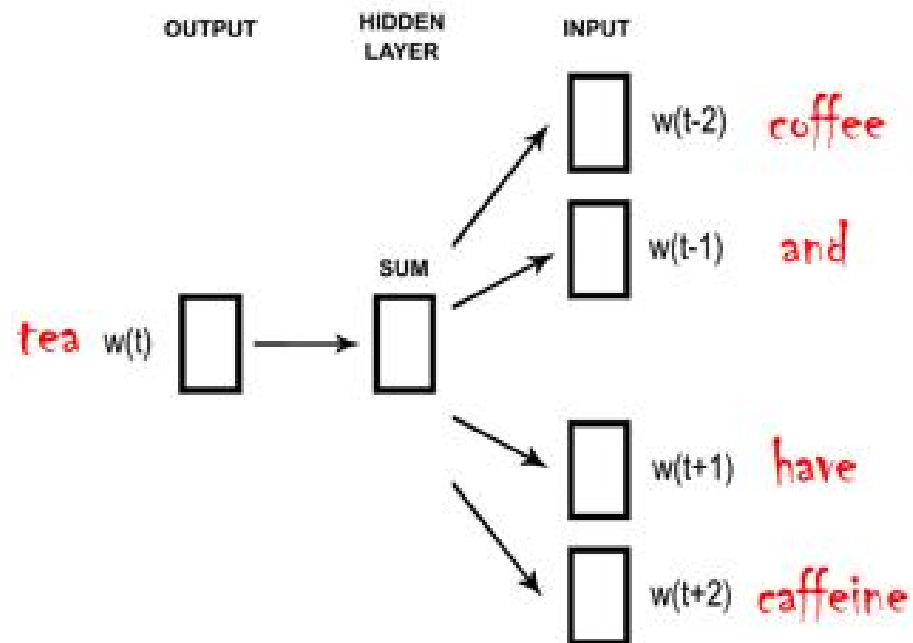
A cup of **tea**
A cup of **coffee**
Tea or **coffee**?
Coffee and **tea** have caffeine
Let's go for a **coffee**
Let's get a **tea**
Coffee vs **Tea**: Which is Best?
I avoid adding sugar to my **tea**
I drink **coffee** with two spoons of sugar



- 因此，可以**从大量现有的句子中学习**保留语义的嵌入

Skip-gram

- 两个基本方法
 - 1) Skip-grams
 - 2) 连续词袋 (CBOW)
- Skip-gram: 使用中心词预测上下文



- 首先，用一个随机向量 $u \in \mathbb{R}^m$ 初始化每个词的嵌入
- 然后，使用第 t 个词语 w_t 来预测其左侧和右侧的词语。目标函数是

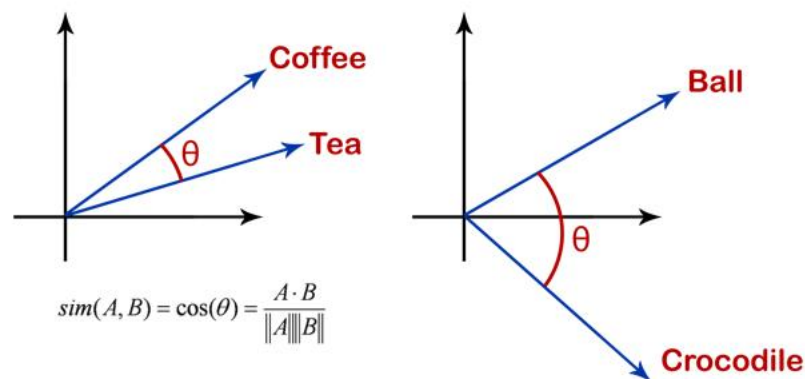
$$J(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} | w_t; \theta)$$

The quick brown fox jumps over the lazy dog.	→	(the, quick) (the, brown)
The quick brown fox jumps over the lazy dog.	→	(quick, the) (quick, brown) (quick, fox)
The quick brown fox jumps over the lazy dog.	→	(brown, the) (brown, quick) (brown, fox) (brown, jumps)
The quick brown fox jumps over the lazy dog.	→	(fox, quick) (fox, brown) (fox, jumps) (fox, over)

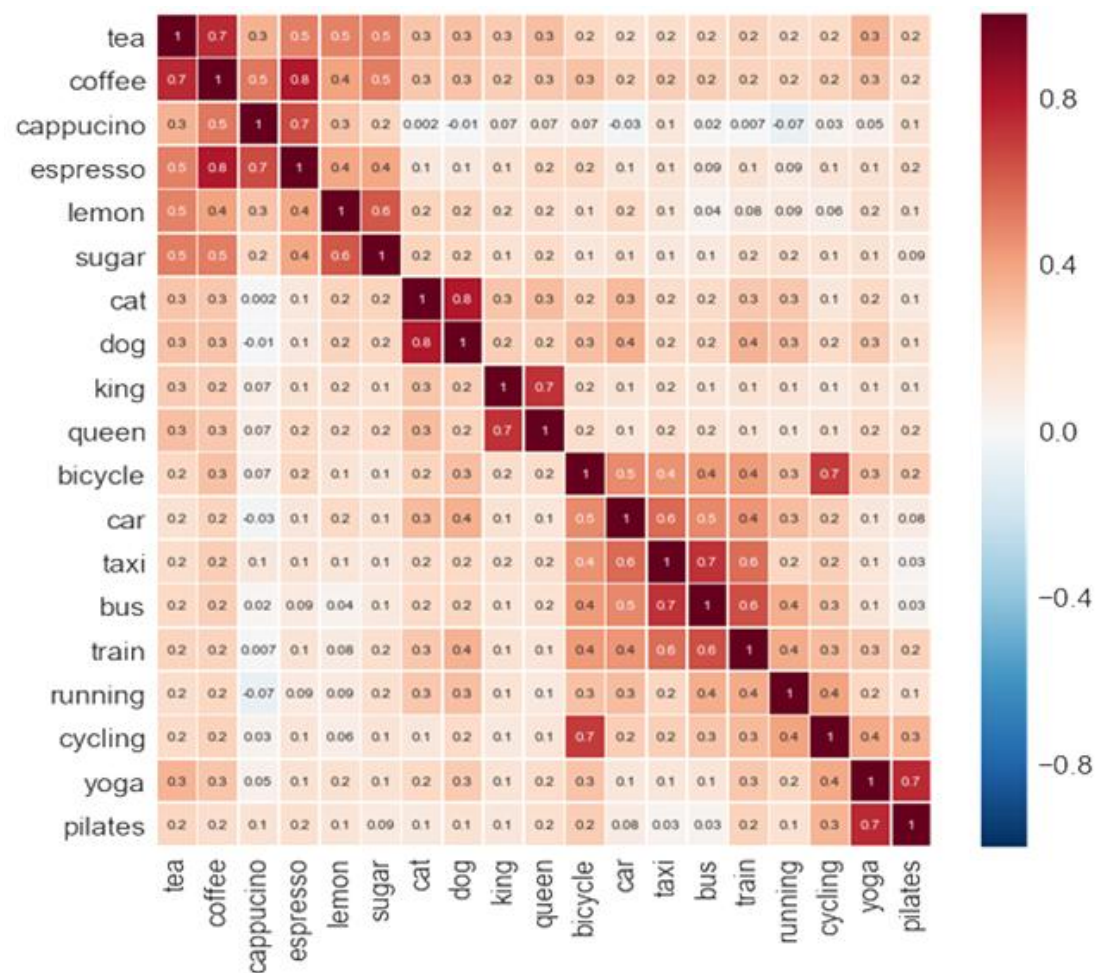
- 预测概率 $P(w_{t+j}|w_t; \theta)$ 建模为

$$P(w_{t+j}|w_t; \theta) = \frac{\exp(u_{w_t}^T u_{w_{t+j}})}{\sum_{w \in \mathcal{V}} \exp(u_{w_t}^T u_w)}$$

- $\mathcal{V}$ 是词汇表中的词语集合
 - u_w 表示词语 w 的嵌入，需要被优化
- 在一个大型语料库上训练之后，单词的语义相似性反映在它们嵌入的余弦相似度中

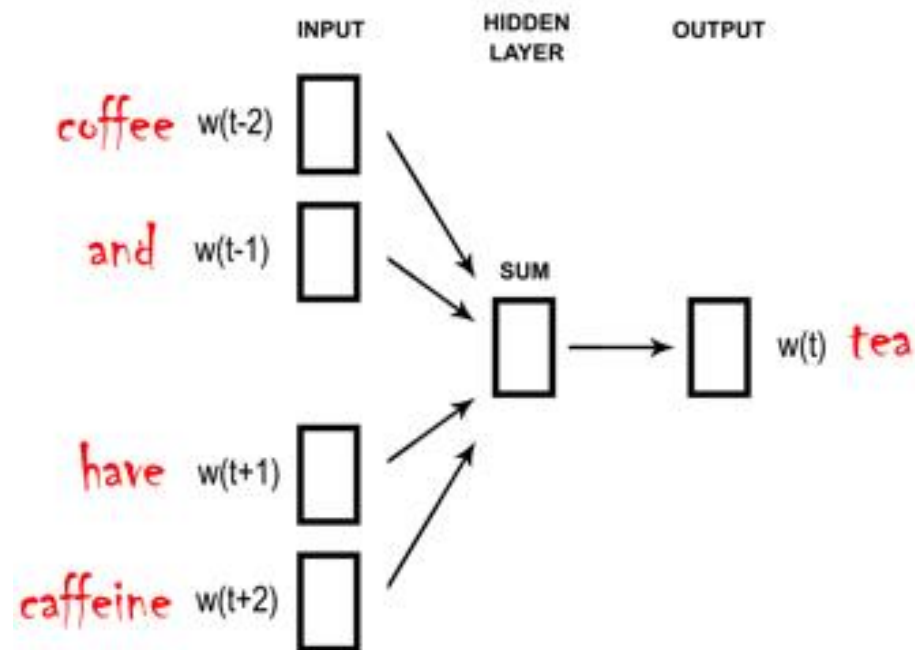


- 在大语料库上训练的词嵌入上评估的余弦相似性示例



连续词袋模型 (Continuous Bag-of-Words)

- 与 skip-gram 不同，CBOW 使用左侧和右侧的词来预测中心词



句子嵌入

- 句子嵌入可以通过多种不同的方式从单词嵌入中获得

1) 平均

Sentence	Embeddings								
coffee	0.2	0.4	0.32	0.89	...	0.77	0.12	0.11	0.99
or	0.54	0.8	0.35	0.34	...	0.56	0.22	0.56	0.43
tea	0.23	0.39	0.55	0.91	...	0.6	0.2	0.61	0.8
?	0.7	0.45	0.56	0.43	...	0.22	0.16	0.33	0.5
Average	0.42	0.51	0.45	0.64	...	0.54	0.17	0.4	0.68

2) 拼接

Sentence

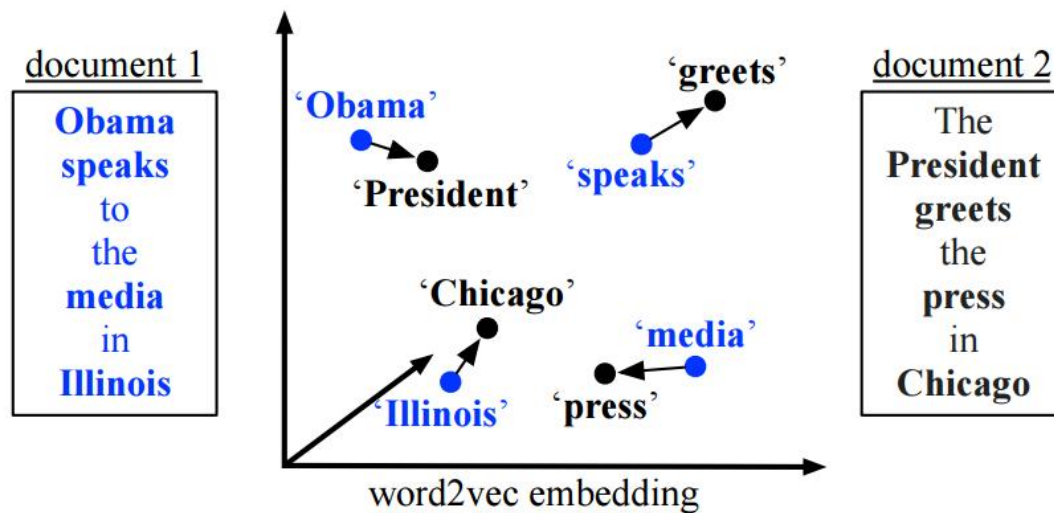
coffee
or
tea
?

Embeddings concatenated

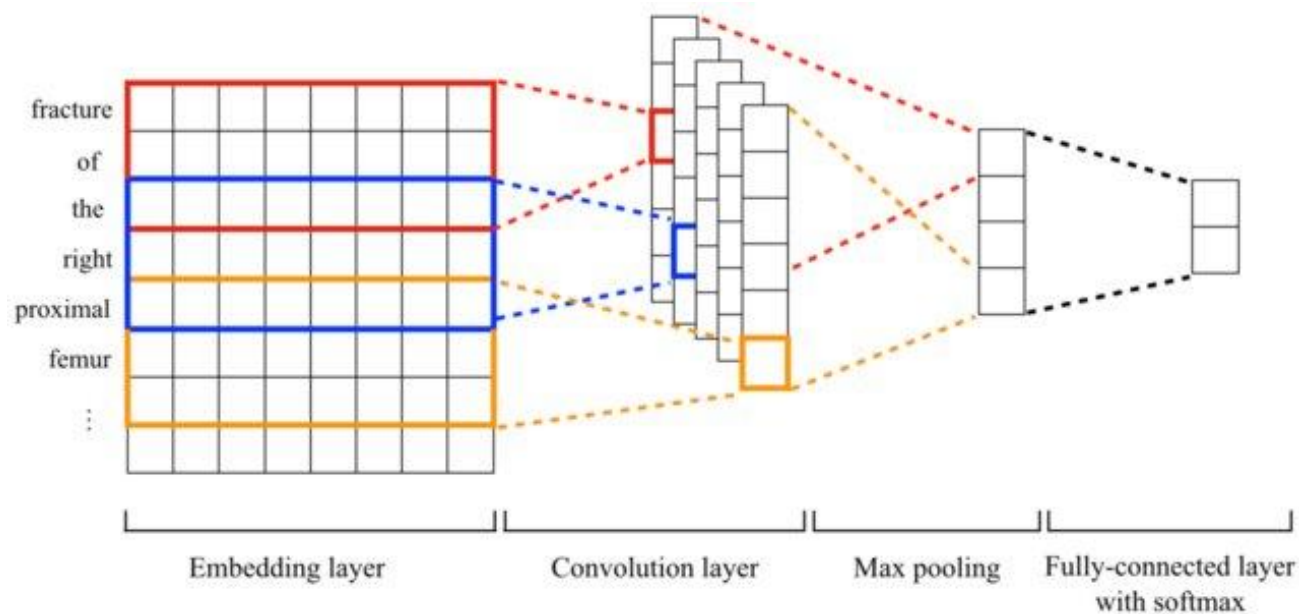
0.2	0.4	0.32	0.89	...	0.77	0.12	0.11	0.99
0.54	0.8	0.35	0.34	...	0.56	0.22	0.56	0.43
0.23	0.39	0.55	0.91	...	0.6	0.2	0.61	0.8
0.7	0.45	0.56	0.43	...	0.22	0.16	0.33	0.5

Dimension of word embeddings

#Tokens

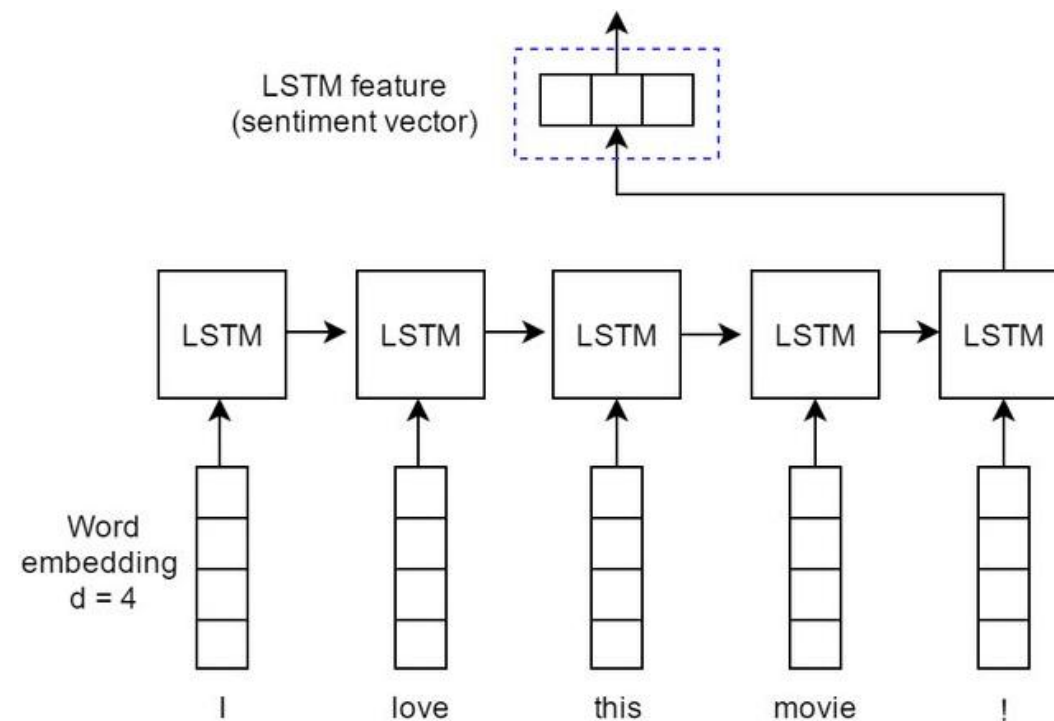


3) 使用 CNN 提取句子嵌入(文本 CNN)



CNN 的参数是使用下游任务优化的

4) 使用 RNN 提取句子嵌入



RNN 的参数是使用下游任务优化的

语境感知的词嵌入

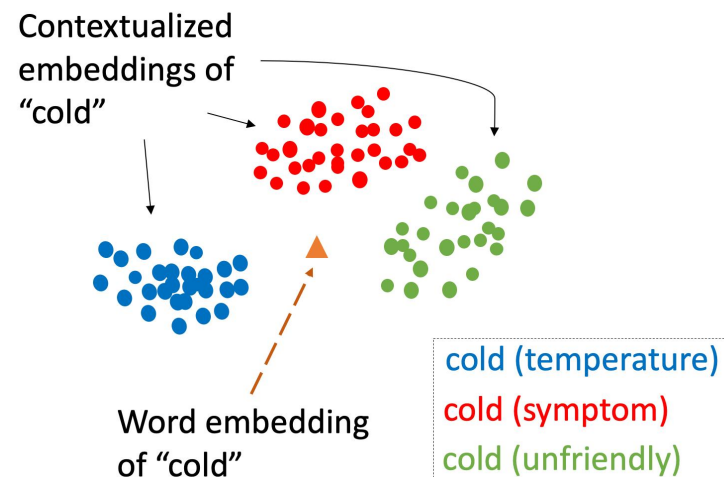
- Word2Vec 嵌入的问题

- Word2Vec 为每个词分配一个**固定的嵌入**
- 但是，单词在不同的语境下通常表现出不同的含义，例如：

1) open a *bank* account

2) on the river *bank*

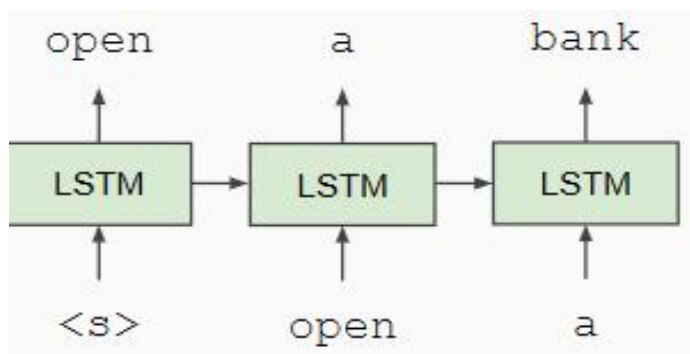
- 单词的嵌入应该根据其**所在的上下文不同而变化**



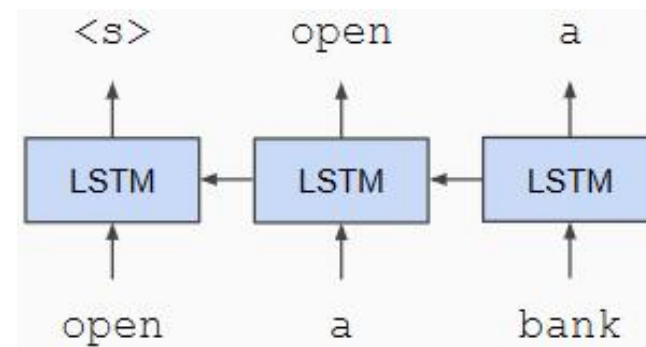
- ELMo

1) 首先，在一个大型语料库上训练一个双向 RNN

2) 然后，将感兴趣的句子通过预训练的 RNN

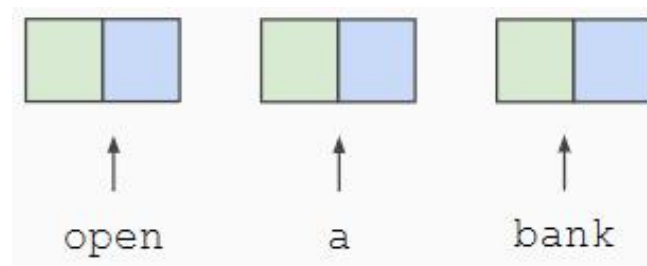


从左到右



从右到左

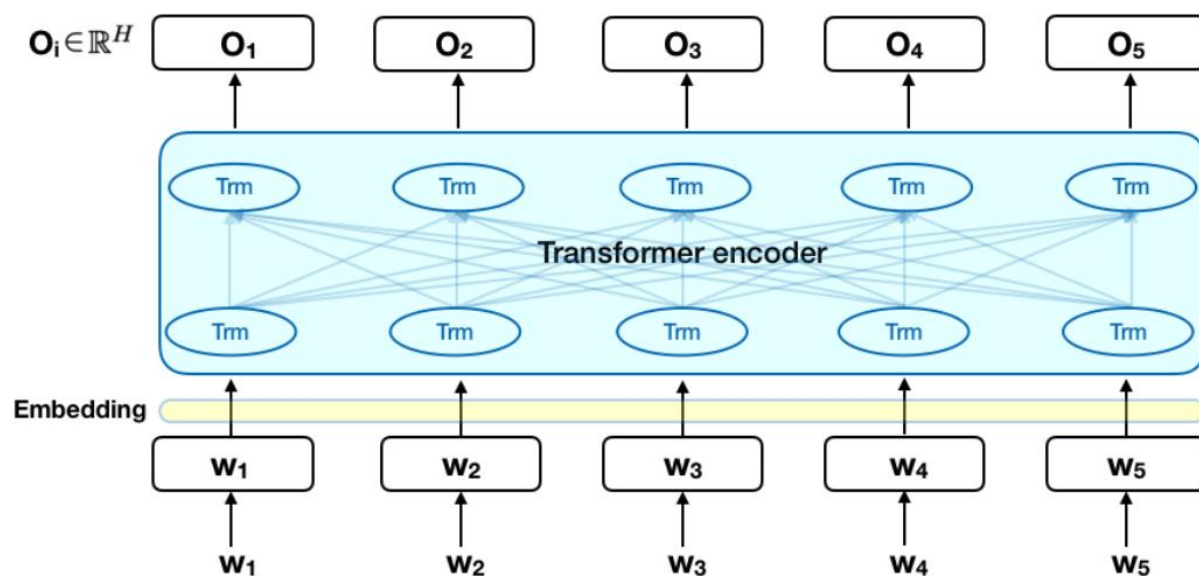
3) 最终的单词嵌入是作为预训练 RNN 的隐藏状态的连接获得的



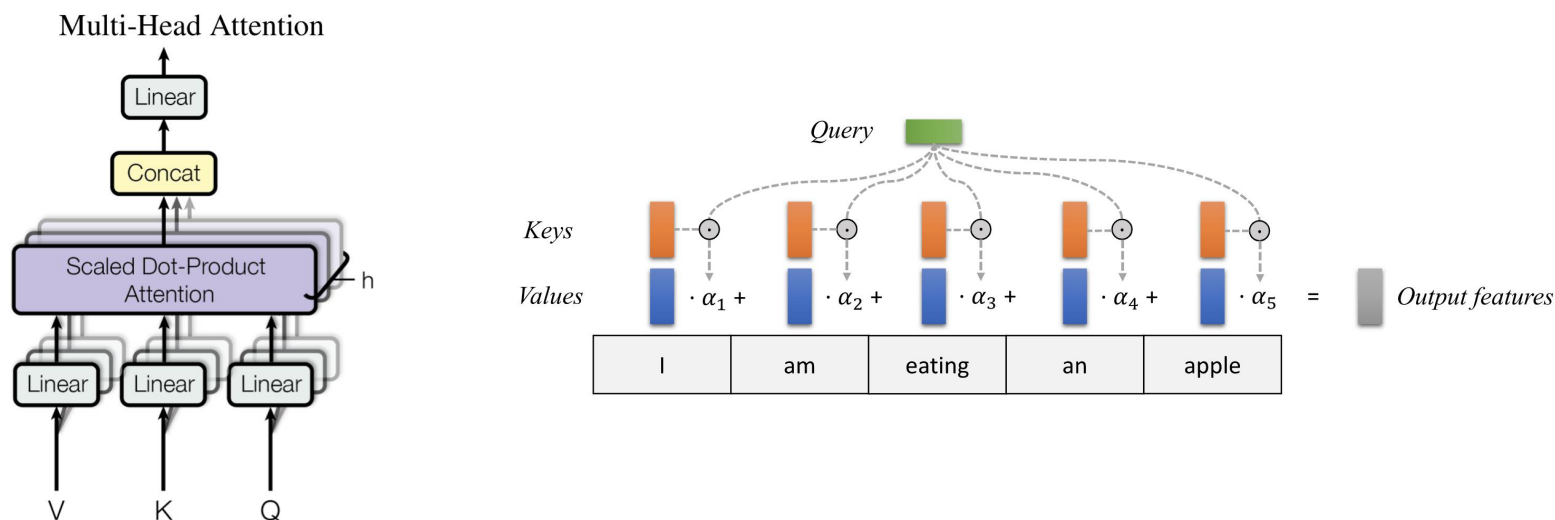
大纲

- 基于词频的文本表示
- Word2Vec词嵌入
- 文本理解的预训练模型BERT
- 基于BERT 微调的文本理解

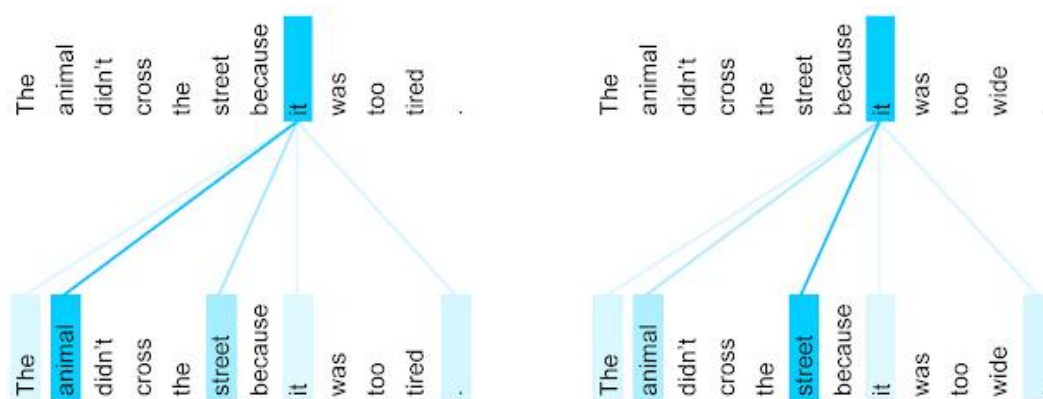
- ELMo 嵌入的问题
 - 1) RNN 难以捕捉文档中词语的长期依赖关系
 - 2) RNN 只允许顺序执行，这使得它难以利用 GPU 中的并行计算资源
- Bidirectional Encoder Representations from Transformers (BERT) 没有使用顺序结构，而是采用基于 Transformer 的“全连接”结构



- 与 MLP 中的全连接不同，这里的全连接结构~~建立~~在一个名为“transformer”的模块上



➤ 例子： 注意单词“it”的相关单词的变化



预训练任务

- 该模型在一个非常大的语料库上进行预训练

1) 预训练任务 1: 掩码语言模型 (Masked Language Model)

- 对于每个输入序列, 随机选择 15% 的 token

- 用 [MASK] 标记替换 80% 的选定 token

went to the store → went to the [MASK]

- 用一个随机词替换 10% 的选定 token

went to the store → went to the running

- 保持剩余 10% 的选定 token 不变

went to the store → went to the store

- 目标：训练 BERT 模型来预测掩码 token 位置的真实 token

store gallon
↑ ↑
the man went to [MASK] to buy a [MASK] of milk

2) 预训练任务 2: 下一句预测 (Next Sentence Prediction, NSP)

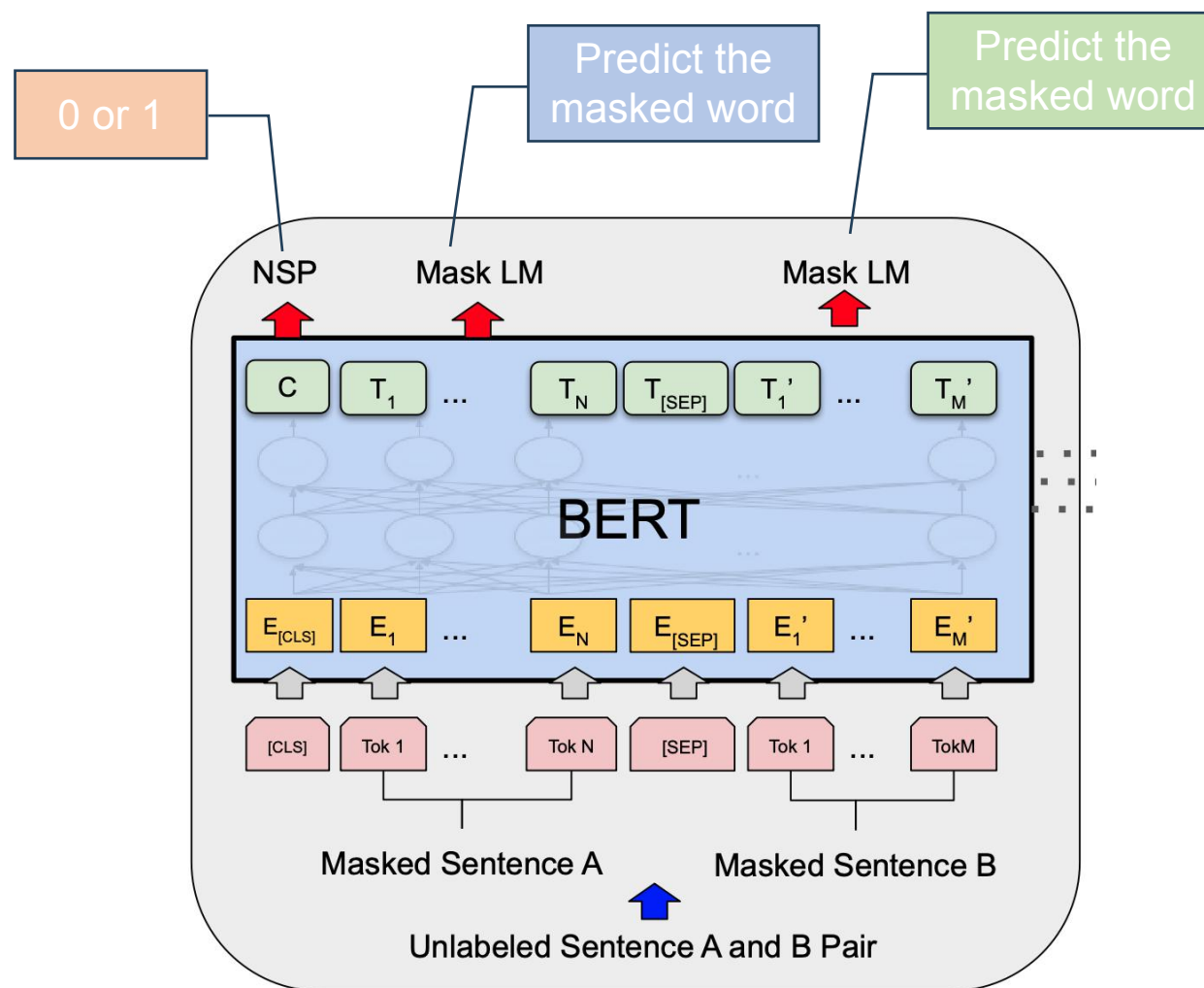
预测句子 B 是否是句子 A 的下一句

[CLS] the man went to [MASK] store [SEP] he bought a gallon [MASK] milk [SEP]

└────────────────────────────────┘ └────────────────────────────────┘
Sentence A Sentence B

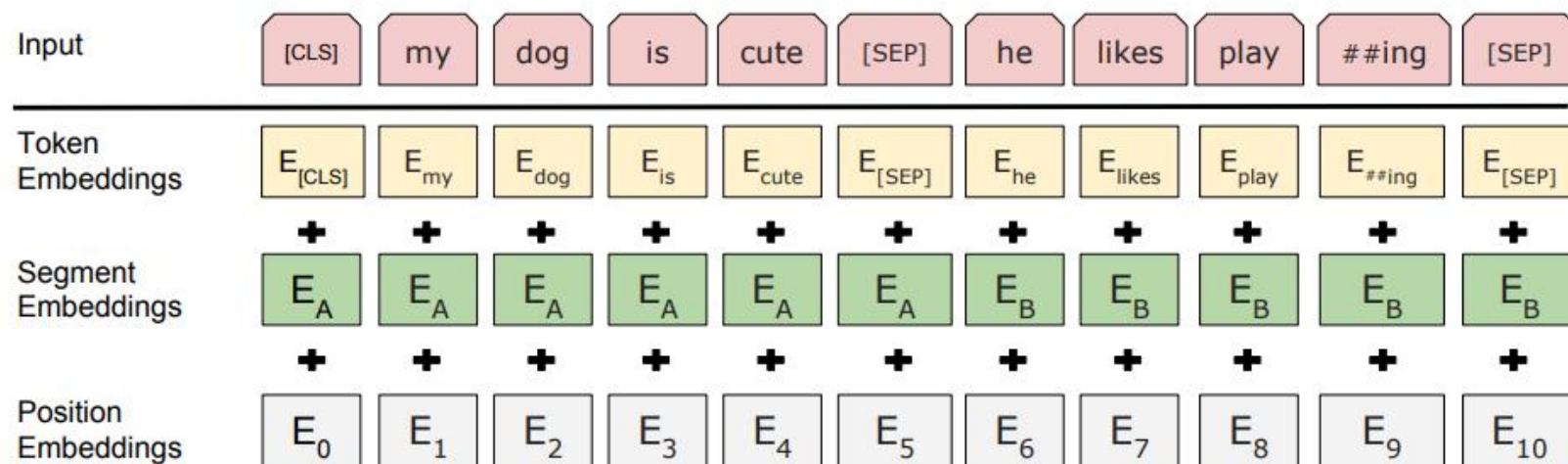
- 50% 的机会让句子 B 成为句子 A 的真实下一句

- 将预训练任务放在一起



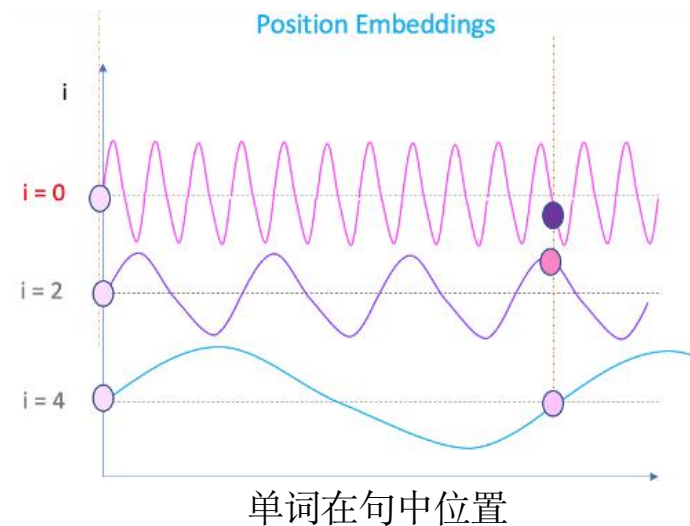
- BERT 模型的输入

将三个 token 嵌入相加作为模型的最终嵌入输入

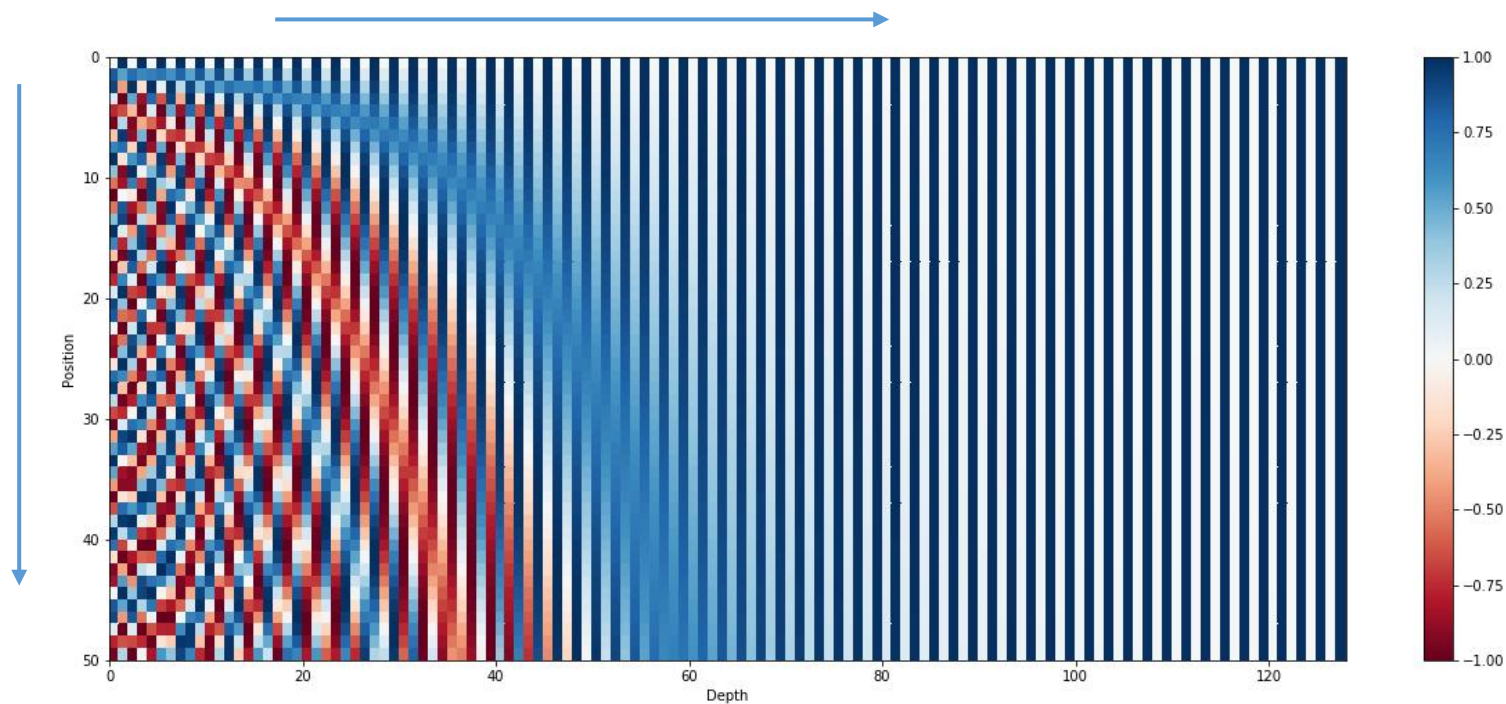


- 位置嵌入

$$PE(pos, i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$



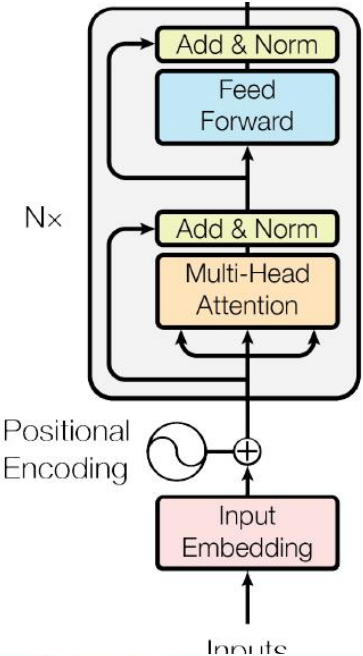
单词位置



- 训练细节

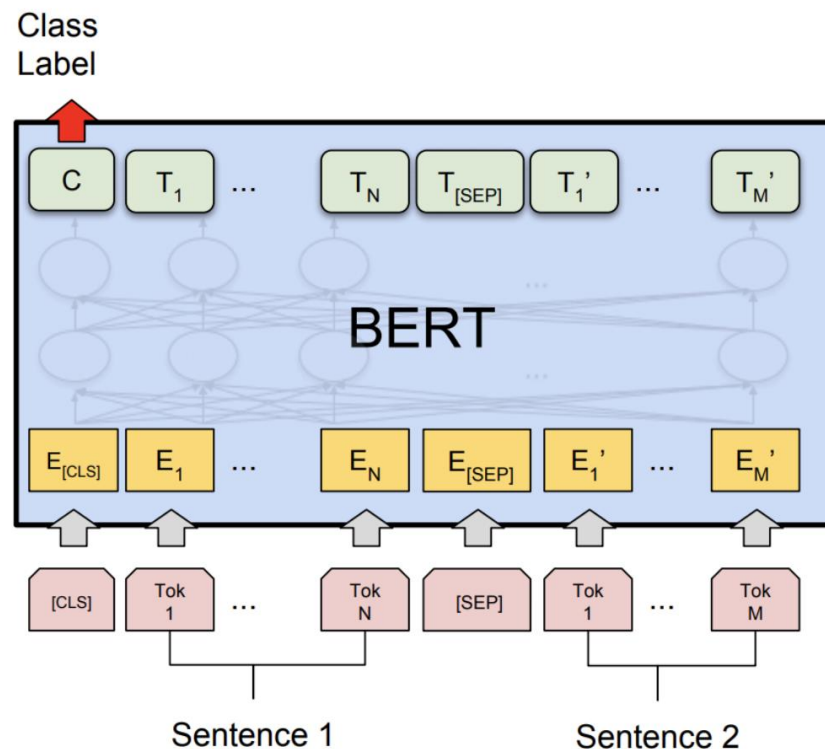
- 训练语料库: Wikipedia (2.5B) + BooksCorpus (0.8B)
- 训练序列长度: 512 个单词片段
- Batch size: 128K
- 训练了一百万步

- BERT 模型的大小
 - BERT-base: 12 层, 768 隐藏层大小, 12 个注意力头, 1.1 亿参数
 - BERT-large: 24 层, 1024 隐藏层大小, 16 个注意力头, 3.4 亿参数



Model	Layers	Width	Heads	Params	Data	Training
Transformer-Base	12	512	8	65M		8x P100 (12 hours)
Transformer-Large	12	1024	16	213M		8x P100 (3.5 days)
BERT-Base	12	768	12	110M	13 GB	
BERT-Large	24	1024	16	340M	13 GB	
XLNet-Large	24	1024	16	~340M	126 GB	512x TPU-v3 (2.5 days)
RoBERTa	24	1024	16	355M	160 GB	1024x V100 GPU (1 day)
GPT-2	48	1600	?	1.5B	40 GB	
Megatron-LM	72	3072	32	8.3B	174 GB	512x V100 GPU (9 days)
Turing-NLG	78	4256	28	17B	?	256x V100 GPU
GPT-3	96	12,288	96	175B	694GB	?
Gopher	80	16,384	128	280B	10.55 TB	4096x TPUv3 (38 days)

- 为了获得单词和句子嵌入，将感兴趣的句子传递给预训练的 BERT 模型



- BERT 的输出是单词和句子的语境化嵌入

大纲

- 基于词频的文本表示
- Word2Vec词嵌入
- 文本理解的预训练模型BERT
- 基于BERT 微调的文本理解

GLUE: 通用语言理解评估基准，包括 6 个句子对和 2 个单句级别任务

- 句子级别任务例子

MNLI

Premise: A soccer game with multiple males playing

Hypothesis: Some men are playing a sport



(entailment, contradiction, neutral)

QQP

Q1: Where can I learn to invest in stocks?

Q2: How can I learn more about stocks?



(duplicate, not duplicate)

SST2

Rich veins of funny stuff in this movie



(positive, negative)

- Token 级别任务

- 问题回答(SQuAD)

Question: The New York Giants and the New York Jets play at which stadium in NYC ?

Context: The city is represented in the National Football League by the New York Giants and the New York Jets , although both teams play their home games at MetLife Stadium in nearby East Rutherford , New Jersey , which hosted Super Bowl XLVIII in 2014 .

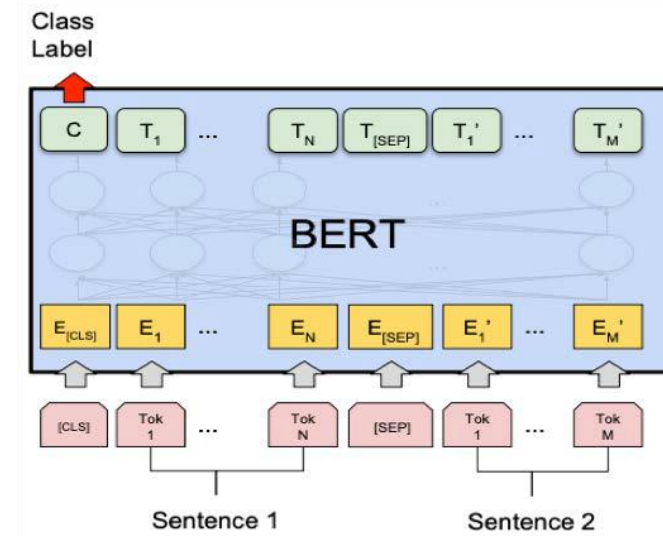
(Training example 29,883)

- 命名实体识别

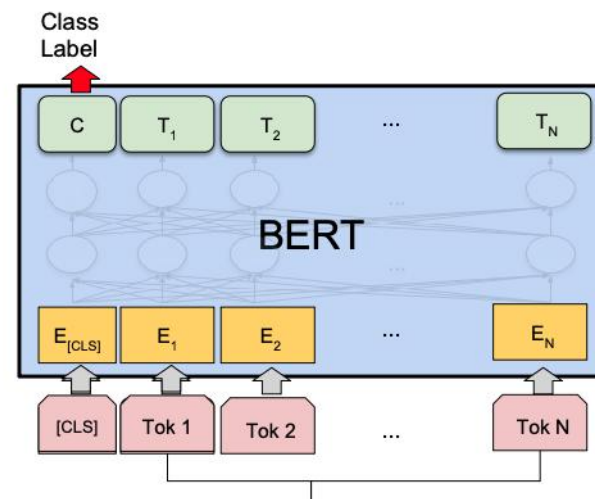
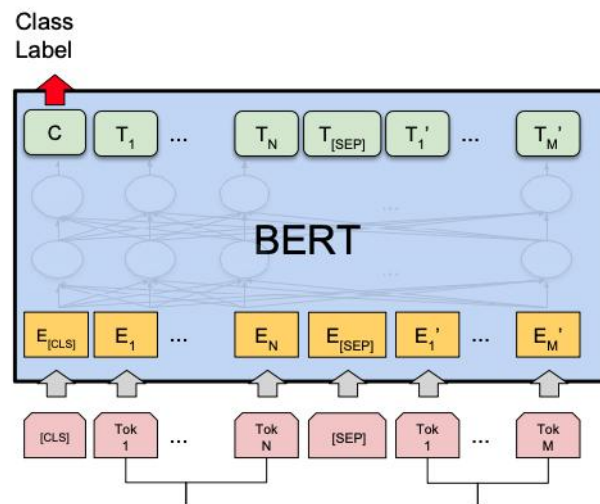
John	Smith	lives	in	New	York
↓	↓	↓	↓	↓	↓
B-PER	I-PER	O	O	B-LOC	I-LOC

- 微调 BERT——对于句子对任务，使用 [SEP] 来分隔带有分段嵌入的两个段落

➤ 在 [CLS] 表示之上添加一个线性分类器



➤ 对于 token 级别预测任务，在输出层的token表示之上添加独立的线性分类器



- GLUE实验结果

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

- 消融研究

➤ 更大的模型通常会带来更好的性能

Hyperparams				Dev Set Accuracy		
#L	#H	#A	LM (ppl)	MNLI-m	MRPC	SST-2
3	768	12	5.84	77.9	79.8	88.4
6	768	3	5.24	80.6	82.2	90.7
6	768	12	4.68	81.9	84.8	91.3
12	768	12	3.99	84.4	86.7	92.9
12	1024	16	3.54	85.7	86.9	93.3
24	1024	16	3.23	86.6	87.8	93.7

课堂小结

- Bag of Words, TFIDF——基于词频的文本表示
- 词嵌入——基于语义的文本表示
- BERT——文本理解的基本模型
- 基于BERT微调范式的各种下游文本理解